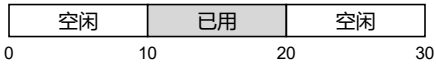
17

空闲空间管理

本章暂时把内存虚拟化的讨论放一放，绕个小弯，谈谈所有内存管理系统都要面对的一个基本问题，不管是 malloc 库（管理进程堆中的页），还是操作系统本身（管理进程地址空间的各个部分）。具体来说，我们要讨论**空闲空间管理**（free-space management）的相关问题。

我们把问题说得更具体一些。管理空闲空间当然可以很容易，我们在讨论**分页**（paging）概念时就会看到。如果你管理的空间被划分成固定大小的单元，事情就很简单：只要用一个列表记下这些固定大小的单元；客户端请求一个单元时，返回列表的第一项即可。

如果你管理的空闲空间由大小可变的单元组成，空闲空间管理就变得更困难（也更有意思）了。这种情况出现在用户级的内存分配库中（比如 `malloc()` 和 `free()`），也出现在用**分段**（segmentation）实现虚拟内存时管理物理内存的操作系统中。无论哪种情况，这里的问题都叫作**外部碎片**（external fragmentation）：空闲空间被切成大小不一的小块，变得支离破碎；即使空闲空间的总量超过了请求的大小，后续的请求也可能因为找不到一块能满足它的连续空间而失败。



上图展示了这个问题的一个例子。这里可用的空闲空间一共有 20 字节；不巧的是，它被分成了两块，每块 10 字节。结果，即使有 20 字节空闲，一个 15 字节的请求也会失败。由此，我们引出了本章要解决的问题。

关键问题：如何管理空闲空间
要满足大小可变的请求，应该如何管理空闲空间？可以用什么策略来尽量减少碎片？各种方法的时间和空间开销如何？

17.1 假设

本章的讨论主要围绕用户级内存分配库中的分配器展开，这些分配器有着悠久的历史。我们借鉴了 Wilson 的出色综述 [W+95]，但也鼓励感兴趣的读者去读原文，了解更多细节[1]。

[1]它将近 80 页长，所以你得真的很感兴趣才行！

我们假设有一个基本的接口，就像 `malloc()` 和 `free()` 提供的那样。具体来说，`void *malloc(size_t size)` 接受一个参数 `size`，即应用程序请求的字节数；它返回一个指针，指向一块这么大（或更大）的区域（这个指针没有特定类型，用 C 的行话来说就是 **void 指针**，void pointer）。与之配套的例程 `void free(void *ptr)` 接受一个指针，释放对应的内存块。注意这个接口隐含的意思：用户释放空间时，不会告诉库这块空间有多大；因此，库必须在只拿到一个指针的情况下，就能算出这块内存有多大。本章稍后会讨论怎么做到这一点。

这个库所管理的空间，历史上称为堆；用来管理堆中空闲空间的通用数据结构，是某种**空闲链表**（free list）。这个结构包含了对被管理内存区域中所有空闲块的引用。当然，这个数据结构不一定真的是链表，只要是某种能记录空闲空间的数据结构就行。

我们进一步假设，主要关心的是前面描述的**外部碎片**。分配器当然也可能有**内部碎片**（internal fragmentation）的问题：如果分配器分出去的内存块比请求的大，那么块中多出来的、没有被请求（因而没有用上）的空间就算作内部碎片（因为浪费发生在已分配单元的内部），这是空间浪费的又一个例子。不过，为简单起见，也因为外部碎片是两种碎片中更有意思的一种，我们主要关注外部碎片。

我们还假设，内存一旦分给客户端，就不能再被重定位到内存中的其他位置。例如，如果程序调用 `malloc()`，拿到了指向堆中某块空间的指针，那么这块内存区域实际上就“归”这个程序所有了（库不能移动它），直到程序通过相应的 `free()` 调用把它还回来。因此，也就没法对空闲空间做**紧凑**（compaction），而紧凑本来有助于对抗碎片[2]。不过，操作系统在实现**分段**时，可以用紧凑来处理碎片（见分段那一章的讨论）。

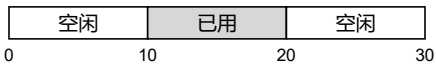
最后，我们假设分配器管理的是一块连续的字节区域。有时，分配器可以要求这块区域增长；比如，用户级的内存分配库在空间用完时，可能会调用内核（通过 `sbrk` 这样的系统调用）来扩大堆。不过，为简单起见，我们只假设这块区域在整个生命周期内大小固定不变。

17.2 底层机制

在深入策略细节之前，我们先介绍大多数分配器都会用到的一些通用机制。首先，我们讨论分割与合并的基础知识，几乎所有分配器都会用到这两种常见技术。其次，我们介绍如何快速、相对轻松地记录已分配区域的大小。最后，我们讨论如何在空闲空间内部构建一个简单的链表，来记录哪些空间是空闲的、哪些不是。

分割与合并

空闲链表包含一组元素，描述了堆中剩余的空闲空间。假设有下面这个 30 字节的堆：



[2] 一旦把指向某块内存的指针交给 C 程序，一般就很难找出指向这块区域的所有引用（指针），它们可能存放在其他变量中，甚至在执行过程中的某一时刻存放在寄存器里。在强类型、带垃圾回收的语言中，情况可能并非如此，因此这些语言可以用紧凑这种技术来对抗碎片。

这个堆的空闲链表上有两个元素。一项描述第一个 10 字节的空闲段（字节 0–9），另一项描述另一个空闲段（字节 20–29）：



如上所述，任何大于 10 字节的请求都会失败（返回 NULL）；根本没有一块这么大的连续内存可用。正好 10 字节的请求，用任意一个空闲块都能轻松满足。但如果请求的大小小于 10 字节，会怎么样呢?

假设我们只请求一个字节的内存。这时，分配器会执行一个称为**分割**（splitting）的操作：它会找到一块能满足请求的空闲内存，把它一分为二。第一块返回给调用者，第二块留在链表上。因此，在上面的例子中，如果请求 1 字节，而分配器决定用链表上两个元素中的第二个来满足请求，那么 `malloc()` 会返回 20（这 1 字节已分配区域的地址），链表最后会变成这样：



从图中可以看到，链表基本没变；唯一的变化是这块空闲区域现在从 21 而不是 20 开始，长度也只剩 9 了[3]。因此，当请求比某个空闲块小时，分配器通常就会用到分割。

许多分配器中还有一个与之相辅相成的机制，叫作空闲空间的**合并**（coalescing）。我们再回到上面的例子（空闲 10 字节、已用 10 字节、再空闲 10 字节）。

对于这个（很小的）堆，如果应用程序调用 `free(10)`，把堆中间的空间还回来，会发生什么？如果我们不假思索地直接把这块空闲空间加回链表，链表可能会变成这样：



注意问题所在：虽然整个堆现在都是空闲的，但它看起来被分成了三块，每块 10 字节。因此，如果用户请求 20 字节，简单地遍历链表是找不到这样一块空闲块的，只能返回失败。

为了避免这个问题，分配器会在释放内存块时合并空闲空间。思路很简单：把一块空闲内存还回去时，仔细查看要归还的块以及附近空闲块的地址；如果新释放的空间正好紧挨着一个（或者像这个例子中那样，两个）现有的空闲块，就把它们合并成一个更大的空闲块。因此，有了合并，最后的链表应该是这样的：



事实上，这正是还没做任何分配时，堆的链表最初的样子。有了合并，分配器能更好地确保应用程序有大块的空闲空间可用。

[3]这里的讨论假设没有头部。这个假设不现实，但我们暂时用它来简化问题。

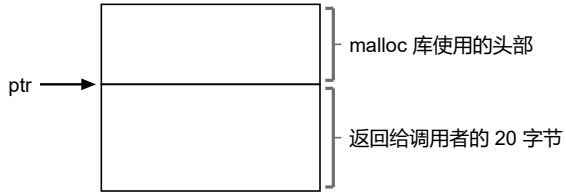


图 17.1：**已分配区域及其头部**

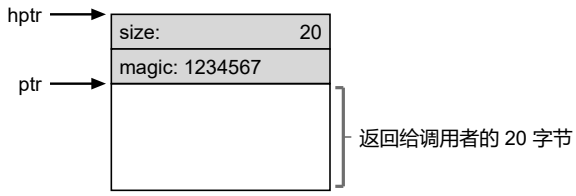


图 17.2：**头部的具体内容**

记录已分配区域的大小

你可能已经注意到，`free(void *ptr)` 这个接口不接受大小参数；这就默认了 malloc 库只凭一个指针，就能迅速确定被释放的内存区域有多大，进而把这块空间放回空闲链表。

为了做到这一点，大多数分配器会在**头部**（header）中存放一点额外的信息。头部保存在内存中，通常就在分出去的那块内存前面。我们再来看一个例子（图 17.1）。在这个例子中，我们看的是一块大小为 20 字节的已分配内存块，由 `ptr` 指向；假设用户调用了 `malloc()`，并把结果存进了 `ptr`，例如 `ptr = malloc(20);`。

头部至少包含已分配区域的大小（这里是 20）；它还可能包含用来加快释放的额外指针、用来做额外完整性检查的幻数（magic number），以及其他信息。我们假设一个简单的头部，它包含区域的大小和一个幻数，如下所示：

```
typedef struct {
    int size;
    int magic;
} header_t;
```

上面的例子如图 17.2 所示。当用户调用 `free(ptr)` 时，库会用简单的指针运算算出头部从哪里开始：

```
void free(void *ptr) {
    header_t *hptr = (header_t *) ptr - 1;
    ...
```

拿到指向头部的指针后，库可以很容易地检查幻数是否与预期值相符，作为一次健全性检查（`assert(hptr->magic == 1234567)`），并通过简单的计算（即把头部的大小加

上区域的大小）算出新释放区域的总大小。注意上一句中这个小而关键的细节：空闲区域的大小是头部的大小加上分配给用户的空间大小。因此，当用户请求 N 字节内存时，库搜索的并不是大小为 N 的空闲块，而是大小为 N 加上头部大小的空闲块。

嵌入空闲链表

到目前为止，我们一直把这个简单的空闲链表当作一个概念上的东西；它只是一个描述堆中空闲内存块的链表。但我们如何在空闲空间本身内部构建这样一个链表呢？

如果是更常见的链表，要分配新节点时，直接调用 `malloc()` 申请节点所需的空间就行了。可惜，在内存分配库内部，你不能这么做！你得在空闲空间本身内部构建这个链表。如果这听起来有点怪，不用担心；它确实有点怪，但还没怪到你做不到的地步！

假设我们要管理一块 4096 字节的内存（即堆的大小是 4 KB）。要把它当作空闲链表来管理，我们首先得初始化这个链表；最开始，链表应该只有一项，大小为 4096（减去头部的大小）。下面是链表节点的定义：

```
typedef struct __node_t {
    int              size;
    struct __node_t *next;
} node_t;
```

现在我们来看一段代码，它初始化堆，并把空闲链表的第一个元素放进这块空间。我们假设堆建在通过系统调用 `mmap()` 获得的一块空闲空间中；这不是构建堆的唯一方法，但在这个例子中很合适。代码如下：

```
// mmap() 返回指向一块空闲空间的指针
node_t *head = mmap(NULL, 4096, PROT_READ|PROT_WRITE,
                    MAP_ANON|MAP_PRIVATE, -1, 0);
head->size   = 4096 - sizeof(node_t);
head->next   = NULL;
```

运行这段代码后，链表里只有一项，大小为 4088。没错，这是个很小的堆，但用来举例正合适。`head` 指针保存着这个范围的起始地址；我们假设它是 16 KB（其实任何虚拟地址都可以）。画出来，堆就是图 17.3 的样子。

现在，假设有人请求一块内存，比如 100 字节。为了满足这个请求，库首先要找一个大到足以容纳这个请求的块；因为只有一个空闲块（大小：4088），所以就选它。然后，这个块会被**分割**成两块：一块大小足以满足请求（以及上面说的头部），另一块是剩下的空闲

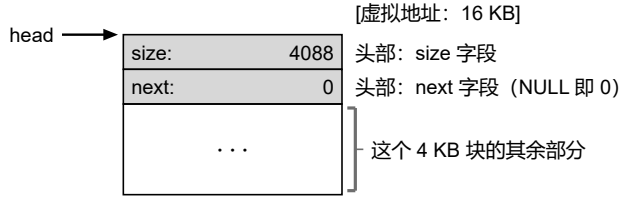


图 17.3：**只有一个空闲块的堆**

 © 2008–23, ARPACI-DUSSEAU

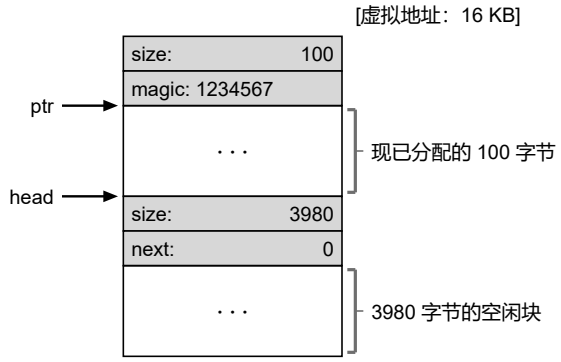


图 17.4：**分配一次之后的堆**

块。假设头部为 8 字节（一个整数表示大小，一个整数表示幻数），堆中的空间现在就像图 17.4 那样。

因此，收到 100 字节的请求后，库从现有的那个空闲块中分配了 108 字节，返回一个指向它的指针（图中标为 `ptr`），把头部信息紧挨着存放在已分配空间前面，供以后 `free()` 时使用，并把链表中那个唯一的空闲节点缩小到 3980 字节（4088 减 108）。

现在我们来看有三块已分配区域时的堆，每块 100 字节（加上头部是 108 字节）。图 17.5 展示了这个堆。

从图中可以看到，堆的前 324 字节现在已经分配出去，因此这段空间里有三个头部，以及调用程序正在使用的三个 100 字节区域。空闲链表依旧平淡无奇：只有一个节点（由 `head` 指向），不过经过三次分割，它现在只剩 3764 字节了。但当调用程序通过 `free()` 归还一些内存时，会发生什么呢？

在这个例子中，应用程序调用 `free(16500)`，归还了中间那块已分配的内存（16500 这个值是这样算出来的：内存区域的起始地址 16384，加上前一块的 108 字节，再加上这一块的 8 字节头部）。这个值在前面的图中用指针 `sptr` 表示。

库立刻算出这块空闲区域的大小，然后把这个空闲块加回空闲链表。假设我们把它插在空闲链表的开头，现在的空间就像图 17.6 这样。

现在，链表的开头是一个小的空闲块（100 字节，由链表头指向），后面是一个大的空闲块（3764 字节）。我们的链表终于不止一个元素了！没错，空闲空间产生了碎片，这很不幸，但很常见。

最后一个例子：假设现在最后两块正在使用的内存也被释放了。如果不合并，最后就会产生碎片（图 17.7）。

从图中可以看到，现在一团糟！为什么？很简单，我们忘了**合并**链表。虽然所有内存都是空闲的，但它被切成了许多块，看起来就像碎片化的内存，尽管实际上并不是。解决方法很简单：遍历链表，把相邻的块**合并**起来；完成后，堆就又完整了。

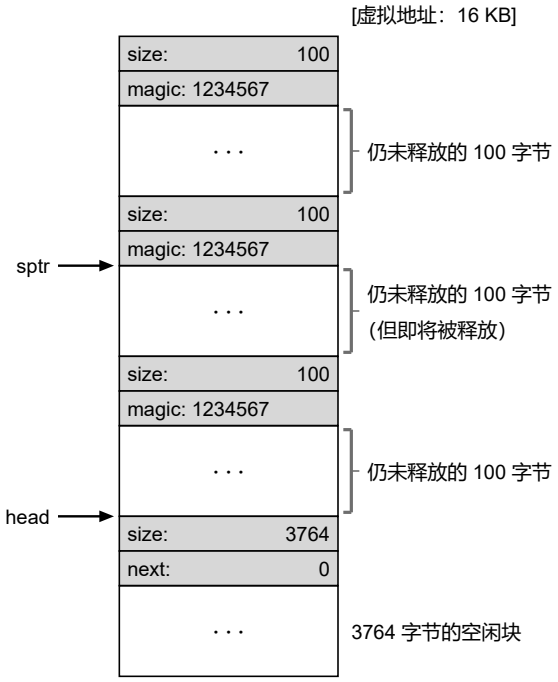


图 17.5：**有三块已分配时的空闲空间**

扩大堆

最后还要讨论许多分配库中都有的一个机制：如果堆的空间用完了，该怎么办？最简单的做法就是直接失败。有时这是唯一的选择，因此返回 NULL 也是一种体面的做法。别难过！你尽力了，虽败犹荣。

大多数传统的分配器一开始只有一个小堆，空间用完后再向操作系统申请更多内存。通常，这意味着它们会发起某种系统调用（例如大多数 UNIX 系统中的 `sbrk`）来扩大堆，再从扩出来的空间里分配新的块。为了处理 `sbrk` 请求，操作系统会找到空闲的物理页，把它们映射到请求进程的地址空间中，然后返回新的堆末尾地址；到这时，就有了一个更大的堆，请求也就能顺利满足了。

17.3 基本策略

既然手头已经有了一些机制，我们来看看管理空闲空间的一些基本策略。这些方法大多相当简单，你自己也能想出来；不妨在往下读之前先试一试，看看能不能想出所有这些方案（甚至想出一些新的！）。

理想的分配器既要快，又要让碎片最少。遗憾的是，由于分配和释放请求的序列可以是任意的（毕竟它们由程序员决定），任何一种策略在遇到不利的输入时都可能表现得很糟。因此，我们不会介绍一种“最好”的方法，而是介绍一些基本方法，并讨论它们的优缺点。

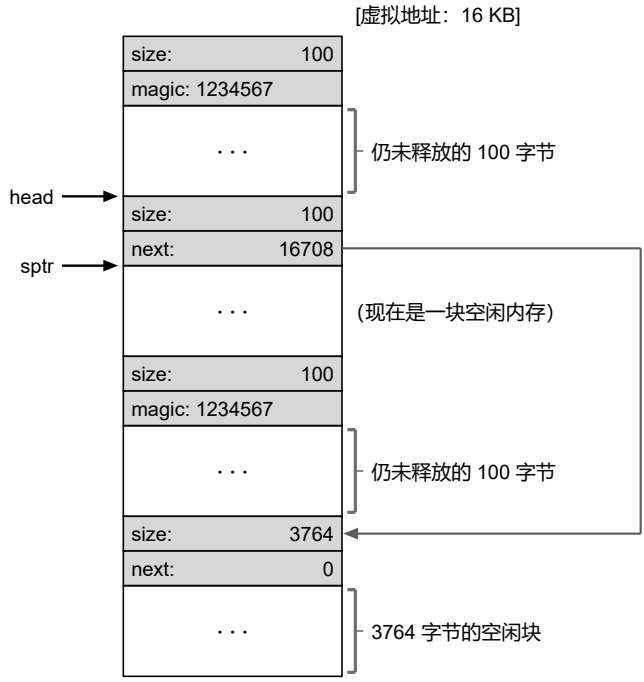


图 17.6：**有两块已分配时的空闲空间**

最佳适应

最佳适应（best fit）策略很简单：首先，搜索空闲链表，找出所有大小等于或大于请求大小的空闲内存块。然后，返回这些候选块中最小的那一块；这就是所谓的最佳适应块（也可以叫最小适应块）。遍历一次空闲链表，就足以找到要返回的那一块。

最佳适应背后的直觉很简单：返回一个大小接近用户请求的块，以此来减少空间浪费。不过，这是有代价的：简单的实现要穷尽搜索才能找到合适的空闲块，性能损失很大。

最坏适应

最坏适应（worst fit）方法与最佳适应相反：找到最大的块，从中分出请求的大小，把剩下的（大）块留在空闲链表上。这样，最坏适应试图留下大的空闲块，而不是像最佳适应那样留下许多小块。不过同样，它需要完整地搜索一遍空闲空间，因此代价可能很高。更糟的是，大多数研究表明它的表现很差，会导致过多的碎片，开销也依然很高。

首次适应

首次适应（first fit）方法直接找到第一个足够大的块，从中分出请求的大小返回给用户。和前面一样，剩下的空闲空间留着供后续请求使用。

 © 2008–23, ARPACI-DUSSEAU

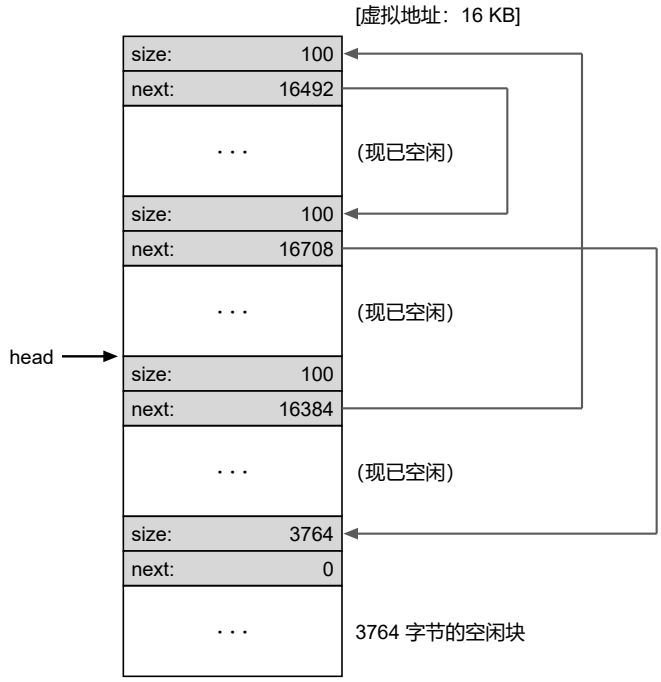


图 17.7：**未合并的空闲链表**

首次适应的优点是快（不需要穷尽搜索所有空闲空间），但它有时会让空闲链表的开头堆满小对象。因此，分配器如何管理空闲链表的顺序就成了一个问题。一种方法是采用**按地址排序**（address-based ordering）：让链表按空闲空间的地址排好序，合并就更容易，碎片也往往会减少。

下次适应

下次适应（next fit）算法不是每次都从链表开头开始首次适应的搜索，而是多维护一个指针，指向上一次查找到的链表位置。这样做的目的是让搜索更均匀地分布在整个链表上，避免链表开头被切得零零碎碎。这种方法的性能和首次适应非常接近，因为同样避免了穷尽搜索。

例子

下面是上述几种策略的几个例子。设想一个空闲链表上有三个元素，大小分别是 10、30 和 20（这里我们忽略头部和其他细节，只关注各种策略如何运作）：



 © 2008–23, ARPACI-DUSSEAU

假设有一个大小为 15 的分配请求。最佳适应方法会搜索整个链表，发现 20 最合适，因为它是能容纳这个请求的最小空闲空间。得到的空闲链表是：



就像这个例子一样，最佳适应方法常常会留下一个小的空闲块。最坏适应方法与之类似，但找的是最大的块，在这个例子中是 30。得到的链表是：



在这个例子中，首次适应策略的结果和最坏适应一样，它找到的是第一个能满足请求的空闲块。区别在于搜索开销：最佳适应和最坏适应都要查看整个链表；首次适应找到一个合适的空闲块就停下，因此减少了搜索开销。

这些例子只触及了分配策略的皮毛。要更深入地理解，还需要结合真实的工作负载和更复杂的分配器行为（例如合并）做更细致的分析。你说，这是不是很适合留作作业？

17.4 其他方法

除了上面介绍的基本方法，人们还提出了大量技术和算法，从各方面改进内存分配。我们在这里列出其中几种供你参考（也就是说，让你的思路不止于最佳适应分配）。

分离空闲链表

有一种由来已久的有趣方法，是使用**分离空闲链表**（segregated lists）。基本思路很简单：如果某个应用程序有一种（或几种）经常请求的大小，就单独维护一个链表，专门管理这种大小的对象；其他所有请求则转发给一个更通用的内存分配器。

这种方法的好处显而易见。专门拿出一块内存来满足某一特定大小的请求，碎片问题就小得多；而且，当请求的大小正好合适时，分配和释放请求都能处理得很快，因为不需要在链表里做复杂的搜索。

就像任何好主意一样，这种方法也给系统带来了新的复杂性。例如，应该拿出多少内存给专门服务特定大小请求的内存池，又该留多少给通用内存池？有一个分配器相当漂亮地解决了这个问题，那就是超级工程师 Jeff Bonwick 为 Solaris 内核设计的 **slab 分配器**（slab allocator）[B94]。

具体来说，内核启动时，它会为那些可能被频繁请求的内核对象（比如锁、文件系统的 inode 等）分配一些**对象缓存**（object caches）；这样，每个对象缓存都是某个特定大小的分离空闲链表，能快速地处理内存分配和释放请求。当某个缓存的空闲空间不足时，它会向一个更通用的内存分配器请求若干内存 **slab**（请求的总量是页大小和相应对象大小的倍数）。反过来，当某个 slab 中所有对象的引用计数都降为零时，通用分配器就可以从专用分配器那里收回这些 slab，这通常发生在虚拟内存系统需要更多内存的时候。

slab 分配器还比大多数分离空闲链表方法更进一步：它让链表上的空闲对象保持在预先初始化好的状态。Bonwick 指出，数据结构的初始化和销毁代价很高 [B94]；slab 分配器

补充：伟大的工程师真的很伟大

像 Jeff Bonwick 这样的工程师（他不仅写了本章提到的 slab 分配器，还领导开发了一个了不起的文件系统 ZFS），是硅谷的核心。几乎每一个伟大的产品或技术背后，都有一个人（或一小群人），他们的天赋、能力和投入都远超常人。正如 Facebook 的 Mark Zuckerberg 所说："在自己岗位上出类拔萃的人，并不只是比相当不错的人好一点点。他们要好上 100 倍。"这就是为什么直到今天，一两个人还能创办一家永远改变世界面貌的公司（想想 Google、Apple 或 Facebook）。努力工作，你也可能成为这样的"100 倍"人才！如果做不到，那就想办法和这样的人共事；你一天学到的，会比大多数人一个月学到的还多。

让链表中被释放的对象保持初始化后的状态，免去了每个对象反复初始化和销毁的过程，从而明显降低了开销。

伙伴分配

由于合并对分配器至关重要，有些方法就是围绕着让合并变简单来设计的。一个很好的例子是二**进制伙伴分配器**（binary buddy allocator）[K65]。

在这样的系统中，空闲内存首先在概念上被看作一整块大小为 2^N 的空间。当有内存请求时，搜索空闲空间的过程会递归地把空闲空间一分为二，直到找到一块大到足以容纳请求的块（再分一次就会太小）为止。这时，就把这个块返回给用户。下面是一个例子：一块 64 KB 的空闲空间在搜索 7 KB 的块时被逐步划分（图 17.8，第 11 页）。

在这个例子中，最左边的 8 KB 块被分配出去（如图中较深的灰色所示），返回给用户；注意，这种方案可能会有**内部碎片**，因为你只能分出大小为 2 的幂的块。

伙伴分配的妙处体现在块被释放的时候。把这个 8 KB 的块还给空闲链表时，分配器会检查它的"伙伴"8 KB 块是否空闲；如果是，就把这两块合并成一个 16 KB 的块。然后，分配器检查这个 16 KB 块的伙伴是否仍然空闲；如果是，就把这两块合并。这种递归的合并过程沿着树一直向上进行，要么恢复出整个空闲空间，要么在发现某个伙伴正在使用时停下来。

伙伴分配之所以效果这么好，是因为确定某个块的伙伴很简单。你可能会问，怎么确定？想想上面空闲空间中各个块的地址。只要仔细想一想，你就会发现每一对伙伴的地址只

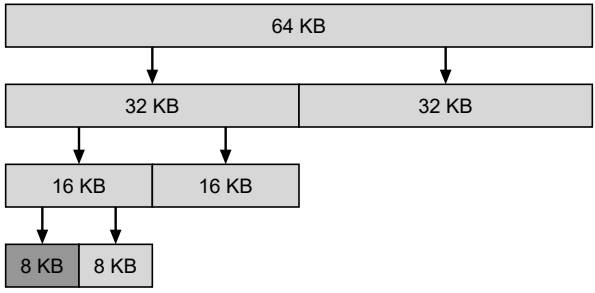


图 17.8：**伙伴系统管理的堆示例**

 © 2008–23, ARPACI-DUSSEAU

差一位；是哪一位，取决于它在伙伴树中的层级。这样，你就对二进制伙伴分配方案的工作原理有了基本的了解。想了解更多细节，照例请参阅 Wilson 的综述 [W+95]。

其他想法

上面介绍的许多方法都有一个大问题：缺乏**可扩展性**（scaling）。具体来说，搜索链表可能相当慢。因此，高级的分配器会使用更复杂的数据结构来降低这些开销，牺牲简单性来换取性能。例子包括平衡二叉树、伸展树或 partially-ordered tree [W+95]。

现代系统通常有多个处理器，运行多线程的工作负载（本书的并发部分会详细介绍这些内容），因此人们花了大量精力让分配器在多处理器系统上运行良好，这并不奇怪。Berger 等人 [B+00] 和 Evans [E06] 给出了两个很好的例子，细节请去看他们的论文。

这只是人们这些年来关于内存分配器的成千上万种想法中的两个；如果你好奇，可以自己去读。如果做不到，至少读一读 glibc 分配器是怎么工作的 [S15]，感受一下真实世界的样子。

17.5 小结

本章讨论了内存分配器最基本的形式。这样的分配器无处不在：它们被链接进你写的每个 C 程序，也存在于为自身数据结构管理内存的底层操作系统中。和许多系统一样，构建这样的系统需要做许多权衡；你对分配器面对的具体工作负载了解得越多，就越能有针对性地调优，让它在这种负载下工作得更好。在现代计算机系统中，构建一个快速、节省空间、可扩展、并且能在各种工作负载下都表现良好的分配器，仍然是一个持续的挑战。

参考文献

[B+00] "Hoard: A Scalable Memory Allocator for Multithreaded Applications" by Emery D. Berger, Kathryn S. McKinley, Robert D. Blumofe, Paul R. Wilson. ASPLOS-IX, November 2000. *Berger and company's excellent allocator for multiprocessor systems. Beyond just being a fun paper, also used in practice!*

[B94] "The Slab Allocator: An Object-Caching Kernel Memory Allocator" by Jeff Bonwick. USENIX '94. *A cool paper about how to build an allocator for an operating system kernel, and a great example of how to specialize for particular common object sizes.*

[E06] "A Scalable Concurrent malloc(3) Implementation for FreeBSD" by Jason Evans. April, 2006. http://people.freebsd.org/~jasone/jemalloc/bsdcan2006/jemalloc.pdf. *A detailed look at how to build a real modern allocator for use in multiprocessors. The "jemalloc" allocator is in widespread use today, within FreeBSD, NetBSD, Mozilla Firefox, and within Facebook.*

[K65] "A Fast Storage Allocator" by Kenneth C. Knowlton. Communications of the ACM, Volume 8:10, October 1965. *The common reference for buddy allocation. Random strange fact: Knuth gives credit for the idea not to Knowlton but to Harry Markowitz, a Nobel-prize winning economist. Another strange fact: Knuth communicates all of his emails via a secretary; he doesn't send email himself, rather he tells his secretary what email to send and then the secretary does the work of emailing. Last Knuth fact: he created TeX, the tool used to typeset this book. It is an amazing piece of software*[4].

[S15] "Understanding glibc malloc" by Sploitfun. February, 2015. sploitfun.wordpress.com/2015/02/10/understanding-glibc-malloc/. *A deep dive into how glibc malloc works. Amazingly detailed and a very cool read.*

[W+95] "Dynamic Storage Allocation: A Survey and Critical Review" by Paul R. Wilson, Mark S. Johnstone, Michael Neely, David Boles. International Workshop on Memory Management, Scotland, UK, September 1995. *An excellent and far-reaching survey of many facets of memory allocation. Far too much detail to go into in this tiny chapter!*

[4] Actually we use LaTeX, which is based on Lamport's additions to TeX, but close enough.

作业（模拟）

程序 `malloc.py` 可以让你探索本章描述的那种简单空闲空间分配器的行为。基本用法详见 README。

问题

1. 首先，用选项 `-n 10 -H 0 -p BEST -s 0` 运行，生成一些随机的分配和释放操作。你能预测 alloc()/free() 会返回什么吗？你能猜出每次请求之后空闲链表的状态吗？随着时间推移，你注意到空闲链表有什么变化？
2. 使用最坏适应策略搜索空闲链表（`-p WORST`）时，结果有什么不同？哪些东西变了？
3. 使用首次适应（`-p FIRST`）呢？使用首次适应时，什么变快了？
4. 对于上面这些问题，链表的排序方式会影响某些策略找到空闲位置所需的时间。使用不同的空闲链表排序方式（`-l ADDRSORT`、`-l SIZESORT+`、`-l SIZESORT-`），看看这些策略和链表排序方式是如何相互作用的。
5. 空闲链表的合并可能相当重要。增加随机分配的次数（比如增加到 `-n 1000`）。随着时间推移，较大的分配请求会怎么样？分别在合并和不合并的情况下运行（即不带和带 `-C` 选项）。你看到结果有什么不同？每种情况下空闲链表随时间变得有多大？这时链表的排序方式有影响吗？
6. 把已分配比例 `-P` 调到 50 以上会怎么样？接近 100 时分配情况会怎样？接近 0 时呢？
7. 你可以发出什么样的特定请求，来产生高度碎片化的空闲空间？使用 `-A` 选项创建碎片化的空闲链表，看看不同的策略和选项如何改变空闲链表的组织方式。